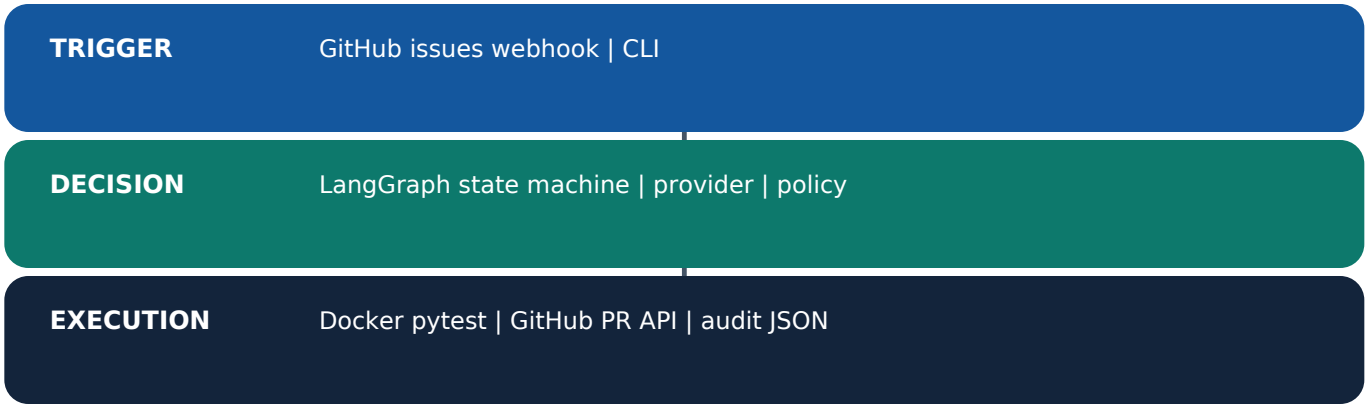
ISSUEWEAVER LAB

From GitHub Issues to Test-Gated Pull Requests

A visual, beginner-friendly engineering manual for an MS in AI portfolio project.



Boundaries protect code, credentials, and default branch separately.

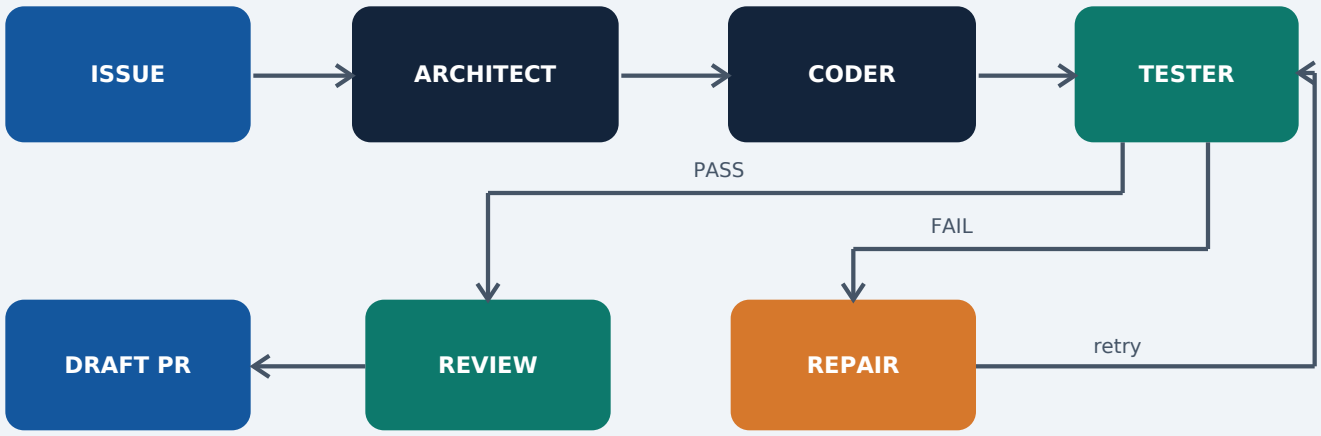
Goal: understand every decision in an Architect -> Coder -> Tester -> Repair -> Reviewer -> PR workflow, not merely copy commands.

What you can prove today	What still requires your environment
37 passing local tests; 91% coverage; deterministic repair demo	Real Docker daemon, actual LangGraph install, OpenAI API call and GitHub PR
Signed-webhook handling, path guards, GitHub REST mocked tests	Public HTTPS webhook endpoint, private repository creation and credentials

Source files and commands: RUN_GUIDE.md and README.md. Original existing repositories are not touched.

01 The big picture

A single model prompt gives a one-time answer. An agent workflow stores state, applies tools, evaluates evidence, and chooses the next action. IssueWeaver separates specialties and makes failures visible.



Only passing tests reach review. Publishing stays off unless explicitly enabled.

Role	Responsibility	Output
Architect	Interprets issue and breaks down work	Plan with up to six tasks
Coder	Proposes complete Python files/tests	Validated JSON file map
Tester	Runs pytest under Docker restrictions	Pass/fail and a bounded log
Repair	Reads failure output and patches files	New candidate + retry counter
Reviewer	Checks tests and write policy	Human-review gate
Publisher	Creates a new branch and draft PR	PR URL, never a merge

02 See the repair loop work

Bundled sample issue: "Normalize repeated whitespace in slugify." The first implementation intentionally contains a defect; the fixture fixes it on its second attempt.

```
# First proposal (fails on repeated spaces)
def slugify(text):
    return text.lower().replace(' ', '-')

# Corrected proposal (after pytest feedback)
def slugify(text):
    return '-'.join(text.lower().split())
```

Edge-case test: slugify(" Hello World ") must equal "hello-world". This demo uses deterministic code, not a real model. Live mode calls an LLM with the same workflow structure.

03 The state machine, in plain English

AgentState is a typed dictionary: issue_title, issue_body, workspace, plan, files, test, attempt, max_repairs, status, trace, and publish. Each graph node returns updated keys rather than starting a new prompt from zero.

```
START -> Architect -> Coder -> Tester
Tester PASS -> Reviewer -> Publisher -> END
Tester FAIL and attempts left -> Repair -> Tester
Tester FAIL and no attempts left -> Stop -> END
```

Default max_repairs = 2 means at most 1 original attempt + 2 repairs = 3 test runs. This prevents unbounded agent loops and uncontrolled API usage.

04 How the real LLM is connected

OpenAIProvider asks an Architect to return a JSON rationale/tasks plan. Coder asks for a JSON object with a summary and mapping of relative .py file names to complete file contents. Pydantic checks required fields, allowed lengths, and file counts.

```
{
  "summary": "Add a small utility with tests",
  "files": {
    "utils.py": "def slugify(text):\n    return '-'.join(text.lower().split())"
  }
}
```

Files from the repo, issue descriptions, and test failures are untrusted data. A prompt may ask the model to ignore malicious instructions, but prompt wording is not a security boundary: file policies and sandboxing provide additional checks.

05 Modules you will study

Location	What it does
src/issueweaver/graph.py	LangGraph nodes, edges, conditional repair
src/issueweaver/providers.py	Fixture and real OpenAI provider
src/issueweaver/models.py	Pydantic and TypedDict contracts
src/issueweaver/workspace.py	Safe path/file policy; bounded source snapshots
src/issueweaver/sandbox.py	Docker test runner and trusted local demo
src/issueweaver/github.py	GitHub issue/archive and draft PR REST adapter
src/issueweaver/api.py	FastAPI webhook, HMAC, delivery IDs
src/issueweaver/service.py	High-level issue orchestration and audit JSON
src/issueweaver/cli.py	Demo and live issue commands

06 Sandboxing is not optional

Generated tests are Python programs. They can open files and perform side effects. Never run arbitrary GitHub issue code with the host-only demo runner. The live runner uses a separate Docker image and deliberately does not receive GitHub or model credentials.

Restriction	Why it matters
network_disabled=True	Blocks direct outbound network calls
read_only=True, bind mode=ro	Prevents writes to root filesystem and mounted repo
cap_drop=[ALL], non-root UID	Reduces container privileges
256 MB, 1 CPU, 64 PIDs	Bounds resource usage
45-second timeout, force remove	Stops or cleans up test processes

Important limitation: Docker containers share the host kernel, and access to docker.sock is highly privileged. A stronger production design uses separate short-lived worker VMs.

07 The GitHub issue trigger

When an issue is given the label swarm-ready, GitHub can POST to /webhook/github. Before using the request, FastAPI verifies HMAC-SHA256 with X-Hub-Signature-256, checks the repository allowlist, verifies that the event is an issue-labeled event, and rejects duplicate delivery IDs.

```
expected = 'sha256=' + HMAC_SHA256(secret, raw_body).hexdigest()
hmac.compare_digest(expected, request_signature)
```

The SQLite delivery log survives restarts for deduplication; it is not a durable queue. If the process crashes halfway through a job, a separate recovery worker is needed in a production system.

08 How a draft pull request gets created

The GitHub adapter does not push to main. It fetches the allowed issue and a size-capped ZIP snapshot, proposes changes, runs tests, and only after an explicit publish opt-in performs these Git data calls:

```
1. GET issue and default-branch commit SHA
2. POST blobs for changed source files
3. POST new tree based on the original tree
4. POST commit with original parent SHA
5. POST refs/heads/issueweaver/issue-N-unique
6. POST pulls (draft=true, base=main)
```

This sequence isolates changes in a new branch. The PR still requires human review, CI, and a separate merge decision.

09 First local run: Windows PowerShell

Extract the project ZIP. Open PowerShell inside the IssueWeaver_Lab folder. Install Python 3.11 or newer and make a virtual environment.

```
python --version
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
python -m issueweaver demo
python -m pytest -q --cov=issueweaver
```

If activation is blocked, in that PowerShell session run Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then activate again. The first pip installation needs internet access.

macOS / Linux alternative

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[dev]'
python -m issueweaver demo
python -m pytest -q --cov=issueweaver
```

10 Create the new PRIVATE repo - a required account step

The current ChatGPT GitHub connector can write to accessible repositories but cannot create a brand-new one. This package includes safe GitHub CLI scripts. They stop if the target repo already exists or the folder already has Git history.

```
# Windows PowerShell (from project folder)
gh auth login
.\scripts\publish_private.ps1

# macOS/Linux
gh auth login
bash scripts/publish_private.sh
```

The intended destination is iam-Debabrata/issueweaver-agent-lab (private). These scripts never push code to an existing repository. Their --private and --source/--push flags follow the official GitHub CLI repository-creation flow.

Do not paste GitHub tokens into the repository. You should authenticate with your own account; browser approval cannot be completed by this package.

11 Live model + Docker workflow

The trusted sample demo is distinct from live issue processing. For live mode, install/start Docker and build the separate runner image:

```
docker version
docker build -t issueweaver-pytest:local sandbox/
```

Use a NEW disposable private playground repository. Create .env from .env.example and fill in the exact target repo, a fine-grained token scoped only to it, and your OpenAI API key.

```
GITHUB_REPOSITORY=iam-Debabrata/issueweaver-playground
GITHUB_TOKEN=YOUR_FINE_GRAINED_TOKEN
OPENAI_API_KEY=YOUR_OPENAI_API_KEY
GITHUB_ALLOW_PR=false
```

```
python -m issueweaver issue --number 1
# inspect runs/issue-1.json, then set GITHUB_ALLOW_PR=true
python -m issueweaver issue --number 1 --publish
```

Real runs are not free: GitHub API requires credentials and model API usage may incur separate charges. If tests fail after all retries, NO PR is created.

12 What pytest and CI actually demonstrate

The repository tests both happy paths and refusal paths: valid/forged webhook signatures, event allowlists, path traversal and symlink rejection, test-failure stops, mock GitHub commit flows, Docker security option construction, and the complete fixture repair loop.

Verification category	Present status
Local Python tests	37 passing tests; 91% measured statement coverage
Local fixture demo	Tester FAIL -> Repair -> Tester PASS
Real LangGraph installed and run	Not verified in this environment
Live Docker daemon test	Not available in this environment
Live GitHub PR	Not attempted; new repo cannot be created by connector
CI in actual GitHub repository	Not run until the project is uploaded

Coverage reports whether lines executed, not whether requirements are correct. A fake passing test can still hide broken production behavior. Always review model-written tests and diffs.

13 Practical debugging

Problem	First action
ModuleNotFoundError	Activate venv and pip install -e .[dev]
Docker daemon unreachable	Start Docker Desktop; docker version
Image not found	docker build -t issueweaver-pytest:local sandbox/
GitHub 401/403	Check selected private repo and token scope
HMAC signature failure	Match secret and raw body bytes
No PR created	Check tests, MAX_REPAIRS, GITHUB_ALLOW_PR
Python package missing inside sandbox	Build a custom reviewed runner image
New repo already exists	Choose an unused new repo name; never overwrite

14 Four-week learning plan

Week	Your exercise	Portfolio artifact
1 - Control flow	Trace each node and state update; sketch graph	Explain graph in docs
2 - Security	Add a policy test; model a prompt injection	Threat model + tests
3 - Observability	Measure retries, latency, and model cost	Results CSV + plotted metrics
4 - Evaluation	Run 10 safe synthetic issues; report failures honestly	Reproducible benchmark report

15 Interview questions to prepare

- Why LangGraph? It expresses conditional feedback loops and makes state transitions testable.
- Why no automatic merge? Tests cover only assertions; a human must inspect the generated diff and CI.
- What is your core engineering tradeoff? Smaller allowable edits reduce risk but limit applicability.
- What would you improve first? Durable job orchestration, stronger VM isolation, benchmarks, costs.
- What is measured? Local code coverage and the fixture demo; live agent success is not measured yet.

Portfolio statement you can make now: "I designed and locally tested an agentic GitHub issue-to-draft-PR prototype, with typed file proposals, a bounded repair loop, HMAC verification, CI configuration, and a restricted Docker runner. The real integration is pending live verification."

16 References and responsible next steps

LangGraph StateGraph: reference.langchain.com/python/langgraph/graph/state/StateGraph

GitHub HMAC: docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries

GitHub CLI: cli.github.com/manual/gh_repo_create

Docker SDK: docker-py.readthedocs.io/en/stable/containers.html

Pytest: docs.pytest.org/en/stable/getting-started.html

After GitHub publication: inspect Actions; run a live test in a disposable repo; document actual results; rotate any accidentally exposed secrets; publish only after review.

Keep the claim smaller than the evidence. That is what makes the project credible.